Movie Ticket Booking System - API Contracts

These contracts define the interactions between the Frontend pages and Backend Microservices. By adhering to these, both teams can develop concurrently.

1. Catalog & Search APIs (Catalog Service)

1.1 Search / List Movies, Theatres, and Shows

Used by the **Home/Catalog Page**

- **Method:** GET /api/v1/search
- **Query Params:** query (string, optional), date (YYYY-MM-DD), city (string)
- **Response (200 OK):**


```
{
  "success": true,
  "data": {
    "movies": [
      {
        "movieId": "m1",
        "title": "Avatar 3",
        "genre": "Sci-Fi",
        "thumbnailUrl": "https://example.com/avatar3.jpg"
      }
    ],
    "shows": [
      {
        "showId": "s101",
        "theatreId": "t1",
        "time": "18:00:00",
        "format": "IMAX 3D"
      }
    ]
```

```
  }
}
```

2. Seat Layout & Inventory APIs (Inventory Service)

2.1 View Seat Layout

Used by the **Seat Layout & Selection Page** to fetch current statuses.

- **Method:** GET /api/v1/inventory/shows/{showId}/seats
- **Cache Strategy:** Fetched rapidly via show:{showId}:seats Redis cache.
- **Response (200 OK):**

```
{
  "showId": "s101",
  "seats": [
    { "seatId": "A1", "status": "AVAILABLE", "price": 300 },
    { "seatId": "A2", "status": "LOCKED", "price": 300 },
    { "seatId": "A3", "status": "BOOKED", "price": 300 }
  ]
}
```

2.2 Lock Seats Temporarily

Used when user proceeds to Checkout. Prevents double-booking via SETNX logic.

- **Method:** POST /api/v1/inventory/seats/lock
- **Payload:**

```
{
  "showId": "s101",
  "seatIds": ["A1", "A2"],
  "userId": "u999"
}
```

- **Response (200 OK) - Lock Successful:**

```
{
  "success": true,
```

"lockToken": "lock_12345abc",
"expiresIn": 600,
"message": "Seats successfully locked."
}

- **Response (409 Conflict) - Lock Failed:**

{
"success": false,
"error": "SEATS_ALREADY_LOCKED",
"message": "One or more selected seats are no longer available."
}

3. Booking & Checkout APIs (Booking Service)

3.1 Confirm Booking / Payment Complete

Used to convert a transient lock into a firm database Booking after successful payment.

- **Method:** POST /api/v1/bookings
- **Payload:**

{
"lockToken": "lock_12345abc",
"showId": "s101",
"paymentId": "pay_98765xyz"
}

- **Response (201 Created):**

{
"success": true,
"bookingRefId": "BK-884930",
"status": "CONFIRMED",
"tickets": [
{ "seatId": "A1", "qrCode": "https://example.com/qr/A1" },
{ "seatId": "A2", "qrCode": "https://example.com/qr/A2" }

]

}

3.2 Cancel / Release Lock

Triggered implicitly if TTL expires, or explicitly if user cancels on the checkout screen.

- **Method:** DELETE /api/v1/inventory/seats/lock/{lockToken}
- **Response (200 OK):**

{

"success": true,

"message": "Seat locks released, returned to pool."

}

Requirement Analysis

Movie Ticket Booking System - Requirement Analysis

1. Project Overview

Problem Statement: Design a high-concurrency movie ticket booking system ensuring real-time seat availability and preventing double booking.

Expected Outcome:

- No double booking under high-load scenarios.
- Real-time updates for seat availability and booking statuses.

---

2. Frontend (FE) Analysis

The Frontend needs to handle frequent updates, real-time locking cues, and provide a fast, seamless user experience.

2.1 Defined Pages

- **Home/Catalog Page:**
  - Browse currently playing movies, upcoming movies, and theatres.
  - Search functionality (caching aware) to find shows rapidly.
- **Theatre/Show Selection Page:**
  - Displays listing of theatres playing a given movie with timings and filling-fast status cues.
- **Seat Layout & Selection Page:**
  - Visual representation of the theatre seating map.
  - Real-time color-coded seat statuses: *Available (White), Selected (Yellow), Locked (Grey), Booked (Red)*.
- **Checkout / Payment Page:**

    - Displays a strict lock countdown (TTL timer, e.g., 10:00 mins) to enforce temporary lock policy.
    - Integration with payment gateway UI (Stripe, Razorpay, etc.).
- **Booking Confirmation / Dashboard Page:**
    - Display finalized tickets, QR Codes, and past user booking history.

2.2 FE Contracts & Behaviors

- **Seat Locking:** The FE must request a **Temporary Seat Lock** as soon as the user selects seats and clicks "Proceed".
- **Conflict Handling:** If the lock API returns a conflict, the FE needs to display a "Seats no longer available" alert immediately.
- **Countdown Timer:** The FE handles a dynamic countdown syncing with the Backend's Redis TTL. If the timer hits zero, the FE automatically redirects out of the checkout page, and the seats become visually unselected.

---

3. Backend (BE) Analysis

The Backend is structured to handle high concurrency (<200 ms response time criteria), high availability, and secure transactions.

3.1 Microservices Break-Up

1. **Catalog Service:** Manages search queries, fetching movies, and listing show schedules. This service will be extremely Read-heavy.
2. **Inventory & Lock Service:** The core of concurrency handling. Oversees real-time seat states, manages distributed locks, and synchronizes data with the frontend.
3. **Booking Service:** Coordinates final order creation, synchronizes external payment validation, and handles permanent ticket generation in the Database.
4. **User & Auth Service:** Handles user registration, standard login, JWT tokens, and user profile operations.
5. **Notification Service:** Handles async processing of tickets via Email/SMS securely.

3.2 Database Calls & Design

- **Catalog DB (PostgreSQL / MongoDB - Read Replicas):**
    - Stores Movies, Theatres, and Shows.
    - *Call:* Rapid GET queries to fetch details based on the user's location and preferences.
- **Booking DB (PostgreSQL / MySQL - ACID Compliant):**
    - Uses STRICT transactional properties to perform final booking inserts.
    - *Tables:* Bookings, Tickets, Show_Seats_Status
    - *Call:* BEGIN TRANSACTION -> Insert into Bookings -> Update Show_Seats_Status mapped array in persistent DB -> COMMIT.
    - Database row-level locking ensures that even if Redis fails, the DB enforces unique final constraints on (show_id, seat_id).

3.3 Cache Calls & Concurrency Handling (Redis Usage)

Redis serves as the primary mechanism to enforce the <200ms response time requirement and seamlessly prevent double booking.

- **Search Caching (High Availability):**
  - *Strategy:* GET search:movies:{query} -> Returns a list of matching active catalog data.
- **Seat Layout Cache (Reads):**
  - *Strategy:* GET show:{showId}:seats -> Returns a JSON structure or Bitmap of total seat availability quickly. Allows blazing-fast reads for the seat layout page.
- **Seat Lock Management with TTL (Writes):**
  - When users select seats, the Backend uses SETNX (Set if Not eXists) or a Lua script to ensure atomicity.
  - *Key:* show:{showId}:seat:{seatId}:lock
  - *Value:* {userId}
  - *TTL (Time To Live):* Set to 10 minutes (600 seconds).
  - *Benefit:* If a user abandons the window or a payment failure occurs, the Lock automatically evicts when the TTL expires, cleanly returning the seat to the available pool without any extra polling overhead.

---

4. Final Expected Outcome Map

By enforcing **Redis TTL Locks** on transient interaction states, and **ACID Transactions** on the final state, the architecture directly satisfies the requirement for a high-concurrency environment with <200ms response time while strictly solving the "double-booking" hazard.